

Technical Requirements Document (TRD)

KisanGat

Smart Dairy Management & ERP System

Version: 1.0

Prepared By: Software Architecture Team

Technology Strategy:

- Frontend: React.js
 - Backend: Django REST Framework
 - Database: MySQL
 - Authentication: JWT
 - Deployment: Docker + Nginx
 - Reports: ReportLab + OpenPyXL
-

1. Technical Overview

KisanGat is a web-based Dairy ERP platform designed to digitize dairy operations such as milk collection, inventory management, customer sales, farmer payments, dairy product management, billing, employee records, and business reporting.

The application follows a modern client-server architecture using React.js as the frontend and Django REST Framework as the backend.

The system will initially support a single authenticated administrator responsible for all operations.

2. Technology Stack

Frontend

Framework

React.js

Reason:

- Component-based architecture
- Fast UI rendering
- Large ecosystem

- Easy API integration

UI Library

Material UI (MUI)

Reason:

- Professional design system
- Pre-built enterprise components
- Responsive layouts

State Management

Redux Toolkit

Reason:

- Centralized state management
- Better scalability
- Predictable state updates

Routing

React Router DOM

Reason:

- SPA navigation
- Route protection support

HTTP Client

Axios

Reason:

- Interceptors
- JWT support
- Request/response handling

Charts

Recharts

Reason:

- Lightweight
- Responsive
- Easy dashboard integration

3. Backend Technology

Framework

Django

Reason:

- Rapid development
- Built-in admin panel
- Secure by default

API Framework

Django REST Framework

Reason:

- REST API support
- Serialization
- Authentication integration

Authentication

Simple JWT

Reason:

- Secure token-based authentication
- Stateless architecture

Task Processing (Future)

Celery + Redis

Purpose:

- Report generation
- Scheduled backups
- Notifications

Not required in MVP.

4. Database

Database Engine

MySQL 8+

Reason:

- Reliable
 - ACID compliant
 - Easy deployment
 - Widely used
-

5. System Architecture

Architecture Style:

Three-Tier Architecture

Presentation Layer Business Logic Layer Data Layer

Architecture Flow:

Client Browser ↓ React Application ↓ REST API ↓ Django REST Framework ↓ Business Services ↓ MySQL Database

6. Frontend Architecture

src/

|—— api/ |—— assets/ |—— components/ |—— layouts/ |—— pages/ |—— routes/ |——
services/ |—— store/ |—— hooks/ |—— utils/ |—— validations/ |—— App.jsx

7. Backend Architecture

backend/

|—— accounts/ |—— dashboard/ |—— farmers/ |—— milk_collection/ |—— customers/ |——
sales/ |—— inventory/ |—— payments/ |—— products/ |—— employees/ |—— expenses/ |——
reports/ |—— settings/ |—— common/

Each module will be implemented as an independent Django application.

8. Authentication Design

Authentication Method:

JWT Authentication

Flow:

Admin Login ↓ Validate Credentials ↓ Generate Access Token ↓ Generate Refresh Token ↓ Return Tokens ↓ Authenticated Requests

Protected APIs:

All APIs except Login API.

Token Expiry:

Access Token: 15 Minutes

Refresh Token: 7 Days

9. Core API Design

Base URL

/api/v1

Authentication

POST /auth/login

POST /auth/refresh

POST /auth/logout

POST /auth/change-password

Dashboard

GET /dashboard/summary

GET /dashboard/revenue

GET /dashboard/inventory

Farmers

GET /farmers

POST /farmers

GET /farmers/{id}

PUT /farmers/{id}

DELETE /farmers/{id}

GET /farmers/{id}/ledger

Milk Collection

GET /collections

POST /collections

GET /collections/{id}

PUT /collections/{id}

DELETE /collections/{id}

Customers

GET /customers

POST /customers

GET /customers/{id}

PUT /customers/{id}

DELETE /customers/{id}

GET /customers/{id}/ledger

Milk Sales

GET /sales

POST /sales

GET /sales/{id}

PUT /sales/{id}

DELETE /sales/{id}

Inventory

GET /inventory

GET /inventory/today

GET /inventory/history

Customer Dues

GET /dues

POST /dues/payment

GET /dues/customer/{id}

Farmer Payments

GET /farmer-payments

POST /farmer-payments

GET /farmer-payments/{id}

Products

GET /products

POST /products

PUT /products/{id}

DELETE /products/{id}

Product Sales

GET /product-sales

POST /product-sales

Employees

GET /employees

POST /employees

PUT /employees/{id}

DELETE /employees/{id}

Expenses

GET /expenses

POST /expenses

PUT /expenses/{id}

DELETE /expenses/{id}

Reports

GET /reports/revenue

GET /reports/collections

GET /reports/inventory

GET /reports/dues

GET /reports/payments

GET /reports/download

10. Inventory Calculation Engine

Formula

Remaining Milk

= Opening Stock

+ Today's Collection

- Today's Sales

System Rules:

- Automatic inventory update
- Carry forward stock
- Inventory consistency checks

11. Billing Engine

Supported Bills

Farmer Bill

Customer Bill

Supported Formats

PDF

Printable HTML

Libraries

ReportLab

Purpose:

- Bill generation
- Statements
- Receipts

12. Reporting Engine

Generate:

Daily Reports

Weekly Reports

Monthly Reports

Yearly Reports

Exports:

PDF

Excel

Libraries:

Pandas

OpenPyXL

ReportLab

13. Security Requirements

Authentication

JWT Authentication

Password Storage

bcrypt hashing

Authorization

Authenticated Admin Only

Transport Security

HTTPS

Input Validation

Backend validation

Frontend validation

SQL Injection Protection

Django ORM

XSS Protection

React escaping

CSRF Protection

DRF security middleware

Rate Limiting

Login endpoint throttling

Audit Logging

Track:

Login

Password Changes

Record Updates

Record Deletions

14. Logging Strategy

Logging Framework

Python Logging

Log Levels

INFO

WARNING

ERROR

CRITICAL

Log Storage

Application Logs

Error Logs

Audit Logs

15. Backup Strategy

Database Backup

Daily Automatic Backup

Backup Retention

30 Days

Backup Storage

Cloud Storage

Local Backup

16. Deployment Architecture

Production Environment

Internet ↓ Nginx ↓ React Static Build ↓ Django API ↓ Gunicorn ↓ MySQL

17. Docker Strategy

Containers

Frontend Container

Backend Container

Database Container

Nginx Container

Benefits

Easy deployment

Environment consistency

Scalability

Simplified maintenance

18. Environment Configuration

Development

Local MySQL

Debug Enabled

Staging

Docker Deployment

Debug Disabled

Production

Docker Deployment

HTTPS Enabled

Secure Environment Variables

19. Performance Requirements

Dashboard Load

< 3 Seconds

API Response Time

< 500ms

Report Generation

< 10 Seconds

Concurrent Users

50+

Database Records

100,000+

20. Technical Decisions Summary

Decision	Selected Technology	Reason
Frontend	React.js	Fast and scalable
UI Framework	Material UI	Professional design
State Management	Redux Toolkit	Predictable state
Backend	Django	Rapid development
API Layer	DRF	REST architecture
Authentication	JWT	Secure and stateless
Database	MySQL	Reliable and scalable
Reporting	ReportLab	PDF generation
Excel Export	OpenPyXL	Spreadsheet reports
Deployment	Docker	Portable infrastructure
Reverse Proxy	Nginx	Security and performance

Final Architecture

Frontend (React.js)

↓

REST APIs

↓

Django REST Framework

↓

Business Services Layer

↓

MySQL Database

↓

Reports & Analytics Engine

KisanGat Version 1 is designed as a secure, scalable, maintainable, and production-ready Dairy ERP platform capable of supporting the operational needs of small and medium dairy businesses while providing a foundation for future expansion into mobile applications, multi-user support, and advanced analytics.